03

The architecture you didn't draw

The single diagram every solo build needs. Auth, data, storage, third parties, and where AI tools quietly cut corners.

SUBJECT	ARCHITECTURE · TRUST LINES	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER THREE OF TEN
TIME	A SIX-MINUTE READ	REV	2026.08

§ 03.1

The napkin test

Here is a test that takes ten seconds. Can you draw your app on a napkin? Not the screens. The machinery: where the data lives, what talks to what, and who is allowed to ask for what.

If you can't, you are not behind. You are normal. AI tools build bottom-up: file by file, feature by feature, each one locally sensible. Nobody ever asked you for the top-down view, so it doesn't exist. The app works and you cannot draw it.

That gap stays invisible until the day it doesn't. The first security question from a real customer. The first "can I export my data?" The first bug that lives between two services instead of inside one file. Every one of those is answered with the drawing you don't have.

This chapter is that drawing: five boxes, universal to almost every solo SaaS, plus the specific corners AI tools cut inside each one. It is also your spec's SHAPE section (chapter 2), drawn instead of written.

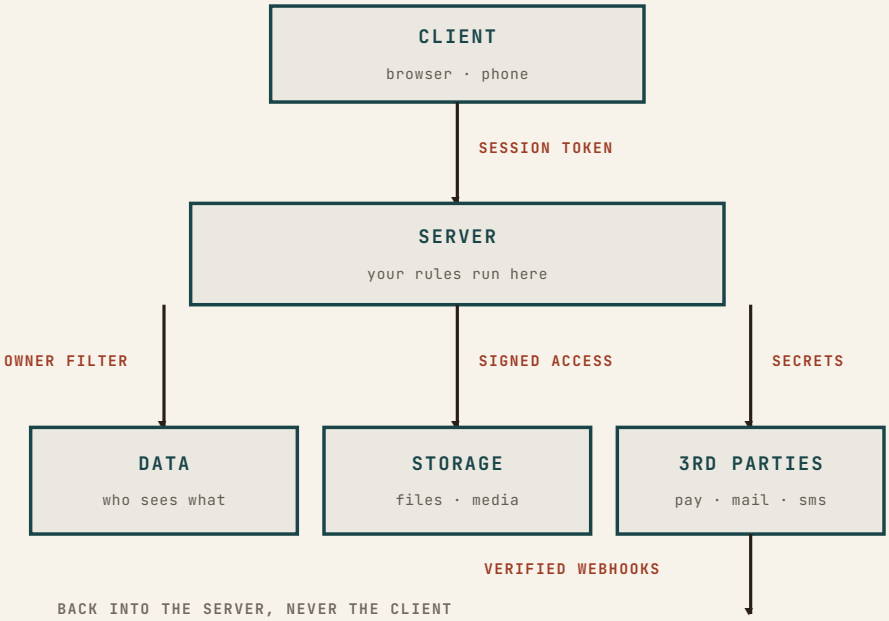
FIELD NOTE

The napkin is also the first thing a buyer, an investor, or a security reviewer asks for. "Walk me through the architecture" is the adult version of "does it work?"

§ 03.2

The five boxes

Nearly every app you will build alone reduces to this map:



Client. The browser or phone. Everything the user touches, and everything an attacker touches too, because the client runs on hardware you do not control.

Server. Your functions, your API routes. The only place where a rule is actually a rule.

Data. The database, and the single most important sentence in your architecture: who is allowed to see which rows.

Storage. Files. Uploads, exports, images. A separate box because it fails separately, and publicly.

Third parties. Payments, email, texts, calendars. Money and messages leave your app here, and truth about money comes back here.

The arrows matter more than the boxes. Every arrow is a trust line, and every trust line needs an answer to one question: what stops the wrong person from using this? Five boxes, five or six arrows, one lock per arrow. That is the whole discipline.

Written down it is a short file, and it ships in the companion pack as `ARCHITECTURE.md`, filled in for the trainer app from chapter two.

§ 03.3

Where the AI cuts corners, box by box

None of what follows is the AI being careless. It is the AI being plausible. Cut corners compile fine and demo fine. They fail only in production, against strangers.

Client: it will trust the browser

The most common corner. Validation that lives only in the form. Prices, quantities, and role flags sent from the client and believed by the server. A “this button is hidden for non-admins” standing in for an actual permission check.

The rule: the client is a suggestion box. Anything it enforces is decoration until the server enforces it again. When you ask the AI for a feature, ask for the server-side check by name, or you will often get the decoration alone.

Server: it will assume your laptop

Code that works locally leans on things production doesn’t have: environment variables that exist only on your machine, file paths, a database seeded by hand. The corner is invisible

because the demo is your laptop. Chapter one's dead-forms story lives in this box, and chapter four is entirely about it.

Data: it will forget who owns the row

Ask an AI for "a function that fetches invoices" and you will frequently get exactly that: all the invoices. The ownership filter, `where user = current_user`, is the single most safety-critical line in your app, and it is precisely the kind of line that drops out when a later session rewrites a query it half-understands.

The strong version of the fix does not trust your queries at all: ownership enforced in the database itself, so a query without the filter returns nothing instead of everything. In Postgres this is row-level security, and it is the difference between "every query must remember the rule" and "the rule is physics."

Storage: it will make the bucket public

Uploads are the classic. The AI wires file upload in one pass, it works, everyone moves on, and the bucket is world-readable because that was the path of least resistance. Photos, PDFs, exports: if the URL works in an incognito window, it works for everyone on earth.

The rule: storage is private by default, and the server hands out temporary signed links. "Can a logged-out stranger open this file's URL?" is a ten-second test. Run it on your own app tonight.

Third parties: it will paste the key where it worked

Two corners here. The first is secrets: API keys belong in environment variables, set on the host, never in code, never in config files the AI writes. The second is truth: when money moves, the payment provider's signed webhook is the fact, and whatever your client says happened is not. Chapter 6 is that story in full.

FIELD NOTE

Ordani runs this way: a birth worker sees her clients and nobody else's, enforced in the database, not in my query discipline. That design has been through two outside security reviews. Chapter 5 builds it step by step.

FROM THE BUILD LOG

ENTRY · 2026-08

The token that was everywhere

Auditing a client project, I searched the repo for anything shaped like a credential. One live API token appeared twelve times, pasted into tool-configuration files, each copy written by an AI session that was quietly being helpful: the command needed the token, the tool saved the command, and nobody looked.

No attacker was involved. No one made a bad decision. Twelve copies of a live key accumulated one convenient moment at a time, in files nobody reads, synced to wherever the repo goes.

The fix took an evening: rotate the key, move it to an environment variable, and add a check that greps every commit for credential-shaped strings. The lesson is chapter one's lesson wearing a security badge: what the machine can check, the machine should check, because you have already proven you won't.

§ 03.4

Make the AI draw it

You do not have to reconstruct the map by hand. The tool that built the accumulation can read it back. Open a session and ask:

> "Read this codebase and produce the five-box architecture map: client, server, data, storage, third parties. For every arrow between boxes, tell me what stops the wrong person from using it. Then list every place a secret lives, with file paths."

Then verify the answer against the code, arrow by arrow, the way chapter one taught you to read diffs: the AI's map is a draft, not a fact. An hour of this is the cheapest security review you will ever run. What you confirm goes into the spec's SHAPE section. What you can't confirm goes on the NOW list, because an arrow you can't explain is work, not trivia.

What you end up with is a file, not a memory. Update it the day an arrow moves.

§ 03.5

Pre-flight: the five locks

PRE-FLIGHT · ONE LOCK PER ARROW — RUN TONIGHT

- [ ] **The client enforces nothing alone.** Every rule the browser applies exists on the server too. Pick your most sensitive action and trace where it's actually checked.
- [ ] **Every query filters by owner.** Better: ownership lives in the database (row-level security), so a forgetful query returns nothing, not everything.
- [ ] **Storage is private by default.** Copy a file URL from your app, open it logged out. If it loads, that's tonight's work.
- [ ] **Secrets live in environment variables on the host.** Grep the repo for anything key-shaped; rotate whatever you find. Then make the grep a pre-release check.
- [ ] **Money truth comes from verified webhooks.** The provider's signature, checked on the server, is the only "it's paid" your app believes.

Five boxes on a napkin, one lock per arrow, drawn once and kept in the repo next to your spec. The next chapter takes this map to the place where it gets stress-tested for real: the day the app leaves your laptop.

NEXT · CHAPTER 04

Deploy day

Environment variables, databases, domains, SSL, secrets. The pre-flight list, in the order things bite you.